

Debugging Assignment - Issues Found & Fixed

While going through the code, I found quite a few bugs - some were silly typos, some were logical errors, and a couple were actually serious security issues. Here's a breakdown of everything that was wrong and how I fixed it.

1. Wrong variable name in GET /users The variable was declared as `allUsers` but the code was trying to send `userList` which didn't exist anywhere. Changed `res.send(userList)` to `res.send(allUsers)`.

2. Type mismatch in GET /users/:id `req.params.id` always comes as a string from the URL, but the users array had numeric IDs. So `u.id === id` was always false. Wrapped it with `Number()` to convert before comparing.

3. Missing return in getUserById() The function was finding the user but never returning it. Added a `return` statement - without it the function was useless.

4. Typo in GET /notes/count `notes.length` - classic typo. Fixed to `notes.length`.

5. Missing await in GET /external-data The function `fetchExternalData()` is async but was called without `await`. This means the response would send a Promise object instead of actual data. Added `await`.

6. Assignment instead of comparison in GET /notes `if (notes = [])` was actually reassigning `notes` to an empty array every time this route was hit - wiping all data. Changed to `notes.length === 0` for proper comparison.

7. Float IDs in generateNoteld() `Math.random() * 1000` gives decimal numbers like `487.3291...` which makes terrible IDs. Wrapped with `Math.floor()` to get clean integers.

8. The function not called for newId `const newId = generateNoteId` was storing the function reference, not the result. Added `()` to actually call it. Also moved ID generation inside the POST route so each new note gets a fresh ID instead of reusing the same one.

9. Wrong logical operator in POST /notes validation `if (!title && !content)` only rejects when BOTH are missing. So a note with just a title and no content would pass through. Changed to `||` so it rejects if either one is missing.

10. String vs number mismatch in DELETE /notes/:id Same issue as #2 - params come as strings. Added `Number()` conversion. Also added a check for when the note isn't found, so it doesn't crash trying to splice at index -1.

11. Wrong variable name in PUT /users/:id `user.name = username - username` was never defined. The destructured variable from req.body was `name`. Changed to `user.name = name`.

12. Assignment in filter in GET /user-notes/:userId `notes.filter(n => n.userId = userId)` - single `=` inside a filter is an assignment, not a comparison. This was setting every note's `userId` to the requested value and returning all notes every time. Changed to `===`.

13. OR instead of AND in POST /login - Security Bug This one was actually dangerous. `if (email === "admin@test.com" || password === "123456")` means anyone who knows just the email OR just the password gets in. Changed to `&&` so both must match.

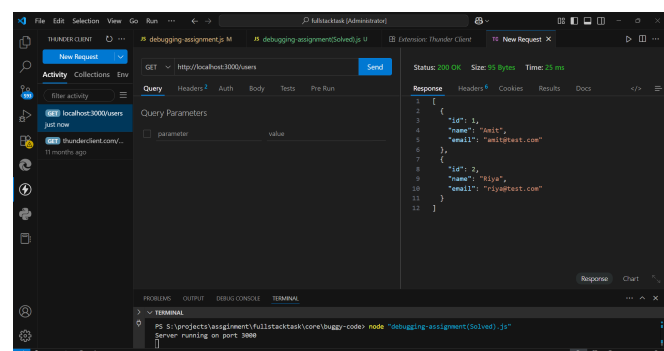
14. filter() instead of find() in GET /profile/:id `.filter()` returns an array, so `user.name` on an array is always `undefined`. Changed to `.find()` which returns the actual object.

15. String concatenation in POST /sum If `a` and `b` come in as strings (which they often do from request body), `a + b` gives "23" instead of 5. Added `Number()` conversion on both.

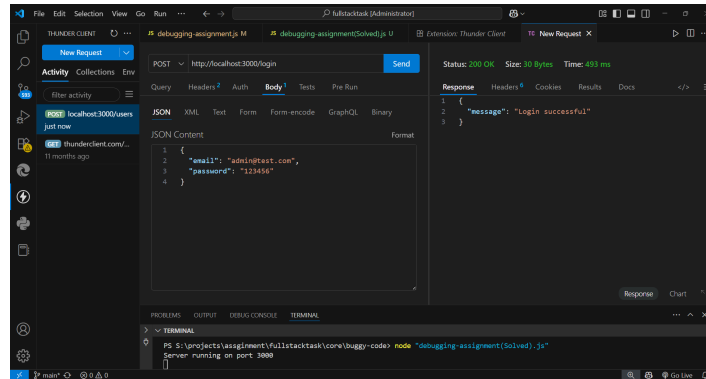
16. Wrong port in console.log The server was listening on port 3000 but the log said "Server running on port 5000". Just a misleading message - fixed it to say 3000.

Test Cases:

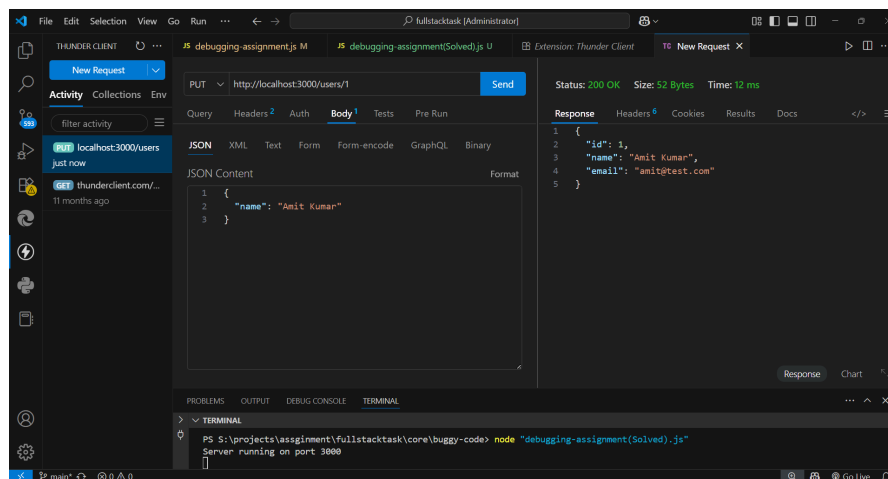
- GET /users** :- Sent request to `http://localhost:3000/users`, got both Amit and Riya in response.



2. **POST /login (valid)** :- Sent POST to <http://localhost:3000/login> with email admin@test.com and password 123456 in body, got "Login successful" in response.



3. **PUT /users/:id** :- Sent PUT to <http://localhost:3000/users/1> with `{ "name": "Amit Kumar" }` in body, user name got updated and returned the updated object.



4. **DELETE /notes/:id** :- Sent DELETE to <http://localhost:3000/notes/1>, got `{ "message": "Note deleted" }` in response and the note was removed successfully.

